Báo cáo kỹ thuật: Dự báo doanh thu cho Vindatathon 2026

Vindatathon 2026 Submission
Mã nguồn: https://github.com/HoangLeminh17/Vindatathon-2026.git
Tài liệu liên quan: `phan2.ipynb`, `phan3.py`, `submission.csv`

Tóm tắt

Báo cáo này tóm tắt cả phần phân tích dữ liệu và phần dự báo doanh thu cho đề bài Vindatathon 2026. Phần EDA cho thấy ba tín hiệu kinh doanh nổi bật: (i) doanh thu có mùa vụ mạnh, trong đó Q2 đóng góp **36.1% doanh thu** và **45.0% lợi nhuận**; (ii) traffic website là chỉ báo vận hành sớm với tương quan tháng $r \approx 0.46$; và (iii) promo đang mua doanh thu bằng lợi nhuận, khi biên lợi nhuận nhóm promo là **-14.5%** so với **+20.0%** ở nhóm không promo. Dựa trên các insight này, mô hình cuối cùng dùng *recursive dual-branch Gradient Boosting*, kết hợp lag theo tuần/năm, rolling means, Fourier mùa vụ và hiệu ứng cuối tháng. Shadow validation theo đúng chiều thời gian đạt $R^2 = 0.7735$ cho Revenue và $R^2 = 0.7662$ cho COGS. Toàn bộ pipeline tuân thủ các ràng buộc về dữ liệu, chống leakage, tính tái lập và khả năng giải thích bằng feature importance.

1 Bối cảnh bài toán và nguyên tắc thiết kế

Đề bài yêu cầu dự báo `Revenue` và nộp `submission.csv` cho giai đoạn **01/01/2023 – 01/07/2024** với ba chỉ số đánh giá là MAE, RMSE và R^2. Tập huấn luyện trong mã nguồn đi từ **04/07/2012 – 31/12/2022** với **3,833** quan sát ngày. Tập test ẩn có **548** ngày, đúng bằng số dòng của `sample_submission.csv`.

Hai nguyên tắc thiết kế chi phối toàn bộ solution là: **(1) chỉ dùng đặc trưng khả dụng tại thời điểm dự báo** và **(2) mô phỏng đúng cách leaderboard sẽ chấm**. Vì vậy, các insight từ `phan2.ipynb` được dùng để định hướng mô hình, nhưng **không phải insight nào cũng được đưa thành feature**. Ví dụ, traffic, promo hay inventory có ý nghĩa kinh doanh rất mạnh, nhưng giá trị tương lai của các bảng này không được quan sát trong hidden test; đưa chúng vào mô hình nộp bài sẽ tạo ra rủi ro không tái lập hoặc leakage theo nghĩa rộng về test-time availability.

2 Trực quan hóa và phân tích dữ liệu

Hình 1 tóm tắt bốn phát hiện quan trọng nhất từ phần EDA. Thứ nhất, nhu cầu có mùa vụ rất rõ: doanh thu trung bình ngày tăng mạnh từ tháng 4 đến tháng 6 và đạt đỉnh vào tháng 5, nên mô hình cần học được cả *year-over-year seasonality* lẫn *local level*. Thứ hai, sessions theo tháng có tương quan dương mức vừa với doanh thu ($r \approx 0.46$), đủ để xem web traffic như một *leading operational indicator* cho lập kế hoạch tồn kho và staffing.

Thứ ba, promo là nguồn *margin leakage* lớn nhất: nhóm không promo tạo biên lợi nhuận gộp khoảng **19.96%**, trong khi nhóm có promo rơi xuống **-14.46%**; tỷ trọng doanh thu

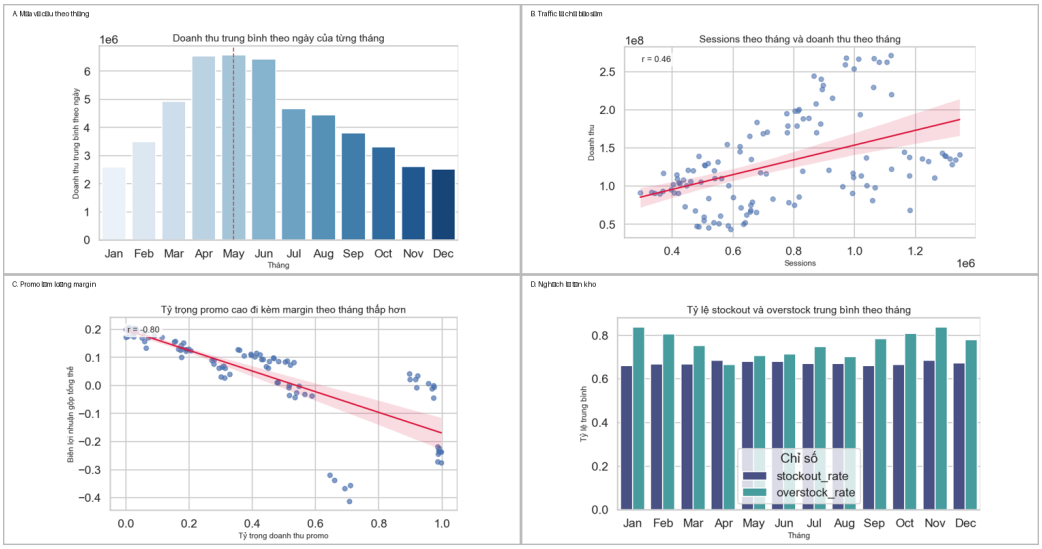

Figure 1: Tóm tắt phần EDA từ `phan2.ipynb`: (A) mùa vụ cầu theo tháng; (B) traffic là chỉ báo sớm với tương quan $r \approx 0.46$; (C) tỷ trọng promo cao đi kèm margin thấp; (D) nghịch lý tồn kho khi stockout và overstock cùng cao.

promo tương quan âm rất mạnh với biên lợi nhuận tổng ($r \approx -0.80$). Điều này cho thấy bài toán tối ưu không phải là tăng doanh thu bằng mọi giá, mà là bảo vệ margin trong các giai đoạn cầu cao. Thứ tư, hệ thống tồn kho đang đồng thời có **stockout rate 67.3%** và **overstock rate 76.4%**, với **39** tháng vừa stockout cao vừa overstock cao; vì thế, trọng tâm là *phân bổ đúng SKU, đúng mùa*, thay vì chỉ tăng tổng tồn kho.

Ngoài bốn phát hiện trong Hình 1, phần EDA còn chỉ ra rằng ba nhóm `wrong_size`, `defective` và `not_as_described` chiếm khoảng **72.6%** gánh nặng hoàn tiền. Trong đó, `Streetwear` và `Outdoor` là hai cụm ưu tiên nếu doanh nghiệp muốn chạy pilot giảm return. Các insight này chủ yếu phục vụ quyết định kinh doanh và hỗ trợ giải thích mô hình, hơn là trở thành feature trực tiếp cho hidden test.

3 Pipeline mô hình và kết quả thực nghiệm

3.1 Feature engineering và kiến trúc mô hình

Mô hình cuối cùng trong `phan3.py` dùng **hai nhánh Gradient Boosting Regressor (GBR)** để xử lý hai bài toán con có đặc tính khác nhau: dự báo `Revenue` và dự báo `cogs_ratio = COGS / Revenue`. Việc dự báo tỷ lệ thay vì dự báo trực tiếp `COGS` giúp mục tiêu ổn định hơn về mặt thống kê; ở bước cuối, `COGS` được khôi phục bằng công thức $\widehat{\text{COGS}} = \widehat{\text{Revenue}} \times \widehat{\text{ratio}}$.

Đặc trưng được tạo hoàn toàn từ `sales.csv` và lịch ngày, gồm bốn nhóm:

- **Calendar**: `dow`, `dom`, `month`, `quarter`, `dayofyear`, `weekofyear`, `is_weekend`, `is_month_start/end`, `days_to_month_end`.
- **Fourier mùa vụ**: `sin/cos` theo chu kỳ năm và bội bậc hai để mô hình hóa đỉnh cầu Q2 và hiệu ứng lặp lại trong năm.
- **Lag features**: nhánh ngắn dùng $7, 14, 28, 56, 364$ ngày; nhánh dài dùng $364, 728$ ngày, phản ánh cả ngắn hạn lẫn cùng kỳ năm trước.
- **Rolling statistics**: nhánh ngắn dùng trung bình trượt $7, 28, 91, 364$ ngày cho cả Revenue và ratio để bắt local level và baseline mùa vụ.

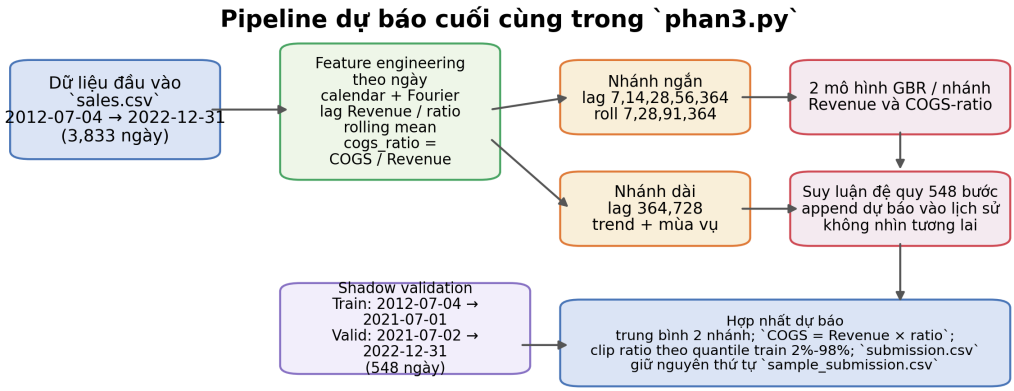

Figure 2: Pipeline dự báo cuối cùng: feature engineering theo ngày, hai nhánh GBR, suy luận đệ quy và xuất `submission.csv` đúng thứ tự `sample_submission.csv`.

Hai nhánh được huấn luyện riêng rồi **lấy trung bình dự báo** để tạo `submission.csv` cuối cùng. Nhánh ngắn mạnh ở tín hiệu gần hiện tại; nhánh dài ổn định hơn khi horizon kéo dài 548 ngày. Suy luận trên hidden test là **recursive multi-step forecasting**: sau mỗi ngày dự báo, giá trị dự báo được append vào lịch sử để tạo feature cho ngày tiếp theo.

3.2 Cross-validation theo thời gian và xử lý leakage

Do dữ liệu là **một chuỗi thời gian hằng ngày duy nhất**, random K-fold là không hợp lệ. Thay vào đó, pipeline sử dụng **shadow validation theo rolling-origin holdout**: lấy đúng **548 ngày cuối** của train làm validation để mô phỏng leaderboard. Split này tạo ra tập train **2012-07-04** → **2021-07-01** với **3,285** điểm và tập valid **2021-07-02** → **2022-12-31** với **548** điểm.

Pipeline chống leakage theo bốn lớp:

- Mỗi dòng feature tại ngày t được sinh từ **lịch sử trước** t; trong mã nguồn, hàm tạo hàng chỉ đọc `history.iloc[:i]`.
- Khi forecast tương lai, mô hình chỉ dùng **các dự báo đã sinh ra trước đó**; không nhìn `Revenue/COGS` thật của test.
- Các biến như traffic, promo hay inventory **không được dùng** trong submission model vì hidden test không cung cấp tương lai của các bảng này.
- Tính tái lập được đảm bảo bằng `np.random.seed(42)`, `random_state=42`, mã nguồn đầy đủ và file nộp đúng format.

Nói cách khác, solution tuân thủ đầy đủ ba ràng buộc loại bài: **không dùng dữ liệu ngoài**, **không dùng Revenue/COGS của test làm đặc trưng** và **có thể tái lập từ mã nguồn**.

3.3 Kết quả shadow validation và giải thích mô hình

Kết quả trên shadow validation được trình bày trong Bảng 1. Mức R^2 khoảng **0.77** cho cả Revenue và COGS cho thấy mô hình bắt được mùa vụ và baseline khá tốt, dù phải dự báo đệ quy trên một horizon rất dài. Trong log chạy được cung cấp, `submission.csv` cuối cùng được tạo từ cấu hình **V1 Improved (Recursive GBR)**; Prophet vẫn xuất hiện trong mã như một baseline tùy chọn, nhưng không phải thành phần bắt buộc của mô hình nộp bài.

Để đáp ứng yêu cầu explainability, báo cáo dùng **feature importance** nội tại của các GBR – một lựa chọn tương đương hợp lệ với yêu cầu "SHAP hoặc feature importance"

Table 1: Shadow validation mô phỏng đúng hidden test horizon (548 ngày).

Target	**MAE**	**RMSE**	R^2
Revenue	558,159	739,187	0.7735
COGS	496,784	661,112	0.7662

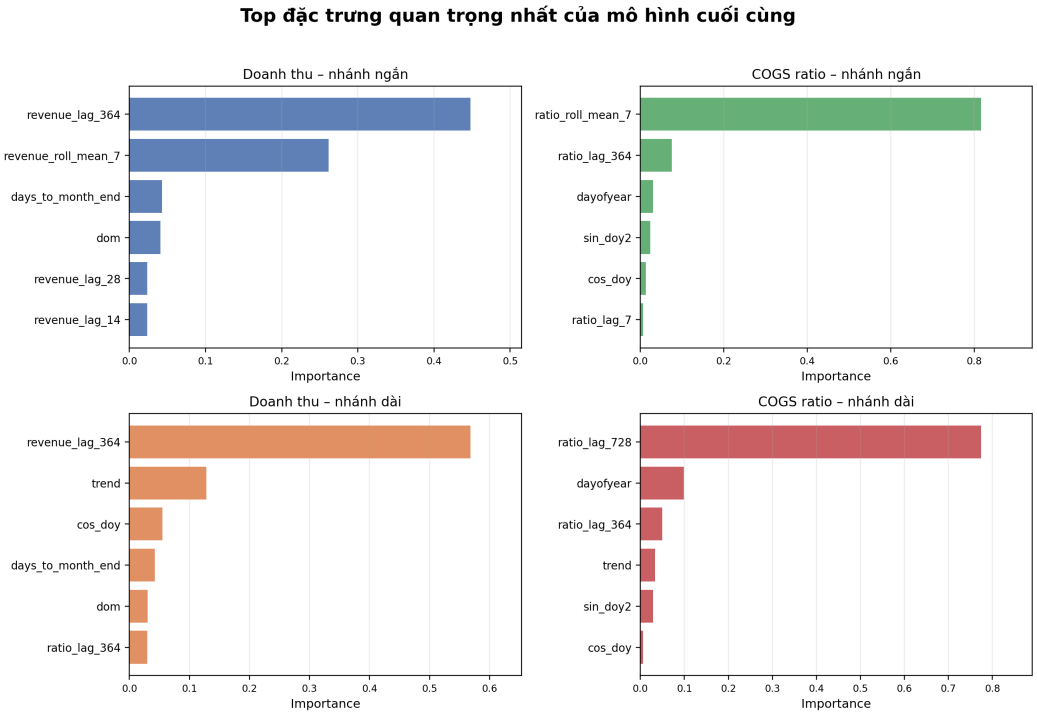

Figure 3: Top đặc trưng quan trọng nhất của mô hình cuối cùng. Revenue phụ thuộc mạnh vào cùng kỳ năm trước và mức cầu gần đây; COGS phụ thuộc chủ yếu vào động học của `cogs_ratio`.

trong rubric. Hình 3 cho thấy `revenue_lag_364` là tín hiệu mạnh nhất cho doanh thu ở cả hai nhánh (0.448 ở nhánh ngắn; 0.568 ở nhánh dài), xác nhận mùa vụ năm trước là trục dự báo quan trọng nhất. Ở nhánh ngắn, `revenue_roll_mean_7` đứng thứ hai, cho thấy local level của tuần gần nhất rất quan trọng trong điều chỉnh forecast ngắn hạn. Các biến `days_to_month_end` và `dom` cũng nổi bật, phản ánh hiệu ứng cuối tháng/đầu tháng khá rõ trong dữ liệu.

Đối với `COGS`, mô hình học chủ yếu trên **ratio features** thay vì mức tuyệt đối: `ratio_roll_mean_7` chiếm 0.817 ở nhánh ngắn, còn `ratio_lag_728` chiếm 0.775 ở nhánh dài. Điều này phù hợp với trực giác kinh doanh: `COGS` biến động cùng `Revenue`, nhưng **cấu trúc cost ratio mượt và bền hơn**, nên dự báo ratio rồi nhân ngược lại là một lựa chọn hợp lý.

4 Kết luận

Solution cuối cùng ưu tiên **đúng bài toán, đúng dữ liệu quan sát được và đúng chiều thời gian** hơn là đẩy thêm nhiều biến ngoại sinh khó kiểm soát. Phần EDA và phần forecast thống nhất với nhau ở ba điểm: mùa vụ Q2 là động lực chính của doanh thu, local level gần hiện tại quyết định dự báo ngắn hạn, và cost structure ổn định hơn khi mô hình hóa dưới

dạng ratio. Với shadow validation đạt $R^2 \approx 0.77$ cho cả Revenue và COGS, pipeline này là một baseline mạnh, dễ tái lập và đáp ứng đầy đủ rubric phần 3 về feature engineering, cross-validation, leakage handling và explainability.

Repository công khai. https://github.com/HoangLeminh17/Vindatathon-2026.git

References

[1] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[2] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.

[3] HoangLeminh17. Vindatathon 2026 repository. https://github.com/HoangLeminh17/Vindatathon-2026.git.